Jenkins shared library: tutorial with examples

Updated: August 15, 2019

You're getting up to speed with Jenkins now and you're mastering the art of the pipeline. But now you've got a few projects up and running, **how do you avoid repeating the same common code across different pipelines?**

I often write code that I know will be reused across several different pipelines in Jenkins. Copying and pasting the same code into different pipelines can quickly become a maintenance headache.

So to avoid this, you store your "reusable bits" (technical term!) in a *Shared Library* in Jenkins. You only need to write some code once, and then you can reference the same code in all of your pipelines.

This Jenkins pipeline shared library tutorial will tell you the basics of setting up a shared pipeline, how to load the library, and how to use it in your pipelines.

What to reuse in Jenkins

If you're already writing code in your pipelines to do anything like this:

- verify the deployment environment
- perform a deployment
- read a configuration file (e.g. a Maven POM file)
- perform a code review of the application (e.g. using a tool like SonarQube)

…then you'll probably be better off putting the code for these into a shared library, so that lots of different projects can use them.

Over time, you'll build up a collection of these reusable functions in your Library.

What is a Shared Library

A shared library is a **collection of independent Groovy scripts** which you pull into your *Jenkinsfile* at runtime.

The best part is, the Library can be stored, like everything else, in a Git repository. This means you can version, tag, and do all the cool stuff you're used to with Git.

> You've already used Groovy in Jenkins. It's a Java-based scripting language, with a very easy, flexible syntax.
>
> Most things you can do in Java, you can do in Groovy. You don't need to be a Groovy expert, but if you want to **start fully learning Groovy, then you can find some learning resources here (http://groovy-lang.org/learn.html)**.

1. First you **create your Groovy scripts** (see below for details on this), and add them into your Git repository.

2. Then, you **add your Shared Library into Jenkins** from the *Manage Jenkins* screen.

3. Finally, you **pull the Shared Library** into your pipeline using this annotation (usually at the top of your *Jenkinsfile*):

```
@Library('your-library-name')
```

What's inside a Shared Library

Inside your Library you'll probably have two types of common code.

- **Steps:** These are called *Global Variables* in Jenkins terminology, but these are the custom steps that you want to be available to all your Jenkins pipelines.

  For example, you might write a standard step to deploy an application, or perform a code review. To do this, add your code into `vars/YourStepName.groovy` and then implement a `def call` function:

```
#!/usr/bin/env groovy
// vars/YourStepName.groovy

def call() {
  // Do something here...
}
```

- **Other common code:** This might include helper classes, or common code that you might want to include inside pipeline steps themselves (meta!). You could also use it as a place to store static constants that you use throughout your pipelines.

  Code like this needs to go in the `src/your/package/name` directory, and then you can use normal Groovy syntax, e.g.:

```
#!/usr/bin/env groovy
// com/cleverbuilder/GlobalVars.groovy
package com.cleverbuilder

class GlobalVars {
   static String foo = "bar"
}
```

  You can then `import` this class into your *Jenkinsfile* and reference the static variable like `GlobalVars.foo`:

Example: Creating and using a Jenkins shared library

Setting up a shared library is fairly easy, and in this section I'll show you how, with a quick example.

1 Create the shared library

First you need to create a Git repository which will contain your library of functions (steps). You can also use Subversion.

In your repository, create a directory called `vars`. This will hold your custom steps. Each of them will be a different `.groovy` file underneath your `vars` directory, e.g.:

```
vars/
    deployApplication.groovy
    parseFile.groovy
    sayHello.groovy
    readSystemCredentials.groovy
    doCodeReview.groovy
```

I'm using Git to store my repository. I've created **a sample repo on GitHub (https://github.com/monodot/pipeline-library-demo)**, which includes one function, the `sayHello` example from the **official Jenkins documentation (https://jenkins.io/doc/book/pipeline/shared-libraries/)**.

See the example repository (https://github.com/monodot/pipeline-library-demo) on GitHub

2 Add your custom steps

Each of your custom steps is a different `.groovy` file inside your `vars/` directory. In Jenkins terminology, these are called *Global Variables*, which is why they are located inside `vars/`.

Create a file for your custom step, and fill in the code. For example, a simple greeting function would look like this:

```
#!/usr/bin/env groovy

def call(String name = 'human') {
  echo "Hello, ${name}."
}
```

Notice how the Groovy script must **implement the `call` method**. Then you should write your custom code within the braces `{ }`. You can also add parameters to your method - the example above has one parameter `name`, with a default value of `human`.

3 Set up the library in Jenkins

Now you've created your library with custom steps, you need to tell Jenkins about it.

You can define a shared library within a *Jenkinsfile*, or you can configure the library using the Jenkins web console. I think it's better to add from the web console, because you then you can share the library across all of your build jobs.

To add your shared library (I'm using my demo on GitHub as an example):

In Jenkins, go to Manage Jenkins → Configure System. Under *Global Pipeline Libraries*, add a library with the following settings:

- Name: `pipeline-library-demo`
- Default version: Specify a Git reference (branch or commit SHA), e.g. `master`
- Retrieval method: *Modern SCM*
- Select the *Git* type
- Project repository: `https://github.com/monodot/pipeline-library-demo.git`

4 Use the library in a pipeline

To use the shared library in a pipeline, you just add `@Library('your-library-name')` to the top of your pipeline definition, or *Jenkinsfile*. Then call your step by name, e.g. `sayHello`:

```
@Library('pipeline-library-demo')_

stage('Demo') {
    echo 'Hello world'
    sayHello 'Dave'
}
```

NOTE: The underscore (_) is **not a typo!** You need this underscore if the line immediately after the `@Library` annotation is not an `import` statement.

If you're using declarative pipeline syntax….

```
libraries {
  lib('pipeline-library-demo')
}

pipeline {

}
```

5 Run the pipeline above, and the output should look something like this:

Jenkins ▸ lib-test ▸ #2

Console Output

```
Started by user Admin
Loading library pipeline-library-demo@master
 > git rev-parse --is-inside-work-tree # timeout=10
Setting origin to https://github.com/monodot/pipeline-library-demo
 > git config remote.origin.url https://github.com/monodot/pipeline-library-demo # timeout=10
Fetching origin...
Fetching upstream changes from origin
 > git --version # timeout=10
 > git fetch --tags --progress origin +refs/heads/*:refs/remotes/origin/*
 > git rev-parse master^{commit} # timeout=10
 > git rev-parse origin/master^{commit} # timeout=10
Cloning the remote Git repository
Cloning repository https://github.com/monodot/pipeline-library-demo
 > git init /Users/tdonohue/.jenkins/workspace/lib-test@libs/pipeline-library-demo # timeout=10
Fetching upstream changes from https://github.com/monodot/pipeline-library-demo
 > git --version # timeout=10
 > git fetch --tags --progress https://github.com/monodot/pipeline-library-demo
+refs/heads/*:refs/remotes/origin/*
 > git config remote.origin.url https://github.com/monodot/pipeline-library-demo # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/monodot/pipeline-library-demo # timeout=10
Fetching upstream changes from https://github.com/monodot/pipeline-library-demo
 > git fetch --tags --progress https://github.com/monodot/pipeline-library-demo
+refs/heads/*:refs/remotes/origin/*
Checking out Revision 5606c9baa9130ec594aec257bbda3d567778715e (master)
Commit message: "First commit"
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 5606c9baa9130ec594aec257bbda3d567778715e
First time build. Skipping changelog.
[Pipeline] stage
[Pipeline] { (Demo)
[Pipeline] echo
Hello world
[Pipeline] echo
Hello, Dave.
[Pipeline] }
[Pipeline] // stage
[Pipeline] End of Pipeline
Finished: SUCCESS
```

TL;DR?

That's it for my intro to Shared Libraries in Jenkins! As you can see they're a useful way to share common code that you might use across different *Jenkinsfiles*.

Here are the important things to remember:

- You need to use Groovy to write your custom functions or steps
- To write pipeline steps that you can call from your pipelines (e.g. `deployApplication`, `readConfigFile`, etc.):
  - Create a file in the `vars/` directory in your repository, with the name of your custom step
  - The file should implement the `def call()` method; you can also define parameters to your step
- To write other common Groovy code, add it into `src/`
- Add your Shared Library in Jenkins using the Configure System screen

Thanks for reading! I hope you've found this example Jenkins pipeline library useful.

How are you using Jenkins pipelines? Any feedback on this article? You're very welcome to post your thoughts in the comments section below.

Join my newsletter

For articles like these, and other good reads, join my newsletter. I send an email once a month, full of awesome stuff for awesome builders.

First name | **Email address** | Subscribe me!

LEAVE A COMMENT

You can use **Markdown** (https://daringfireball.net/projects/markdown/syntax) in your comment. To write code, indent lines by 4 spaces.

11 Comments

Type Comment Here (at least 3 chars)

Name (optional) | E-mail (optional) | Website (optional) | Preview | Submit

Brian • last year

I'm assuming you can also have a common Jenkinsfile pipeline that you can use throughout your builds by setting specific variables that get passed to it?

1 **Reply**

Tom Donohue (https://cleverbuilder.com) • last year

Hi Brian,

Yes you can certain.y have a common pipeline. Just store your pipeline (basically your *Jenkinsfile*) in a Git repo, and then, in your Job configuration, specify this pipeline (choose *Pipeline script from SCM* option). You could define config variables in the Job itself to set the parameters required by the pipeline, e.g.:

- app name
- target environment (dev/test/prod)

and then specify the values for these, when you run the Job.

HTH Tom

1 **Reply**

Chris • last year

You wrote "Pipeline library functions (called steps)". In which type can these functions written, like declarative or scripted pipeline syntax?

-3 1 **Reply**

Anonymous • last year

Nice Article

2 1 **Reply**

Devon • last year

Is there any way to import libraries that are on some other repository without having to define them in Jenkins? In larger legacy projects it's ESPECIALLY annoying to have to wait multiple days for 4 teams to juggle tickets that will then maybe add a single library to Jenkins because you lack access.

1 1 **Reply**

Tom Donohue (https://cleverbuilder.com) • last year

Haha..... I know exactly what you mean.

You can define shared libraries at Folder level. So if you have a folder that you have admin rights on, you can add the shared library there. (New Item › Folder)

If not, I think you can declare a git URL explicitly in your Jenkinsfile, e.g.:

`@Library('git.example.com/yourorg/your-repo@master'` (http://git.example.com/yourorg/your-repo@master')`)`

-3 | **Reply**

Anonymous • 5 months ago

...but in case of @Library('**git.example.com/yourorg/your-repo@master'** (http://git.example.com/yourorg/your-repo@master')) you probably should specify somehow a git-credential... How to do it in this case?

| **Reply**

Tom Donohue (https://cleverbuilder.com) • 5 months ago

I tried this out, and it seems that this feature only works with GitHub, and you can't supply credentials either.

I would recommend adding the shared library explicitly into Jenkins configuration - you can set up the credentials there. Ask your administrator to do this. See the section above "Set up the library in Jenkins"

-1 | **Reply**

Tom Donohue (https://cleverbuilder.com) • 5 months ago

Update - I was wrong on this! To clone from a repository directly, you can use the `library` step at the top of a declarative pipeline, e.g.:

```
library identifier: 'mylibraryname@master',
    //'master' refers to a valid git-ref
    //'mylibraryname' can be any name
    retriever: modernSCM([
      $class: 'GitSCMSource',
      credentialsId: 'your-credentials-id',
      remote: 'https://github.com/monodot/private-jenkins-library.git'
(https://github.com/monodot/private-jenkins-library.git')
    ])
```

Example on GitHub: https://gist.github.com/monodot/f7a2b91efd5c2408eeebbc91fed2e819 (https://gist.github.com/monodot/f7a2b91efd5c2408eeebbc91fed2e819)

Reply

Anonymous • 7 months ago

Very helpful article, thanks a lot.

Reply

Anonymous • 2 weeks ago

Thank you very much for this article. I tried to use the library with the declarative syntax. I could only get it working with:

```
pipeline {
    libraries {
        lib('pipeline-library-demo')
        }
    }
```

-1 Reply